

提示词工程——生成式软件工程第 02 讲（上下文工程、指令遵循与长程任务）

NJU 生成式软件工程课程笔记

2026 年 9 月 2 日



视频作者/频道：[在此填写频道或作者]

发布日期：2026-09

视频时长：1 小时 38 分 26 秒

视频链接：<https://www.bilibili.com/video/BV1CQt365EzW>

PDF 制作 Skill：<https://github.com/wdkns/wdkns-skills>

目录

1 绪论：从 Prompt 到上下文工程	2
2 上下文工程：Agent 的完整视野	2
2.1 解剖一次 Agent 会话的上下文	2
2.2 Agent 上下文中的“喵”测试	3
2.3 学习方式的变革：费曼学习法 × AI	3
2.4 从细节中学习：Codex 安全确认机制的逆向工程	3
3 意图明确化：约束的幂	4
3.1 Intent-Spec-Impl Gap	4
3.2 “一句话生成极品飞车”案例	4
3.3 压缩合法解空间：从“AI 味”到你的品位	5
4 模型如何遵循指令	6
4.1 有限计算与“纸和笔”	6
4.2 强化学习如何塑造“舔狗”行为	7
4.3 注意力机制：指令是抢夺注意力的资源	8
4.4 强化学习带来的副作用	9
4.5 案例：如何用 AI 读论文	9
5 长程任务的建议	10
5.1 为什么大型复杂系统里 AI 反而游刃有余	10
5.2 你仍然需要有概念	12
5.3 一个 Research Failure 案例：符号执行引擎优化	12
5.4 Latency vs Throughput：工作模式的转变	13
5.5 算力换智力：把灵感改造成搜索	13
5.6 大世界假设与未来的危机	15
6 总结与延伸	15
6.1 本讲核心脉络	15
6.2 对软件工程实践的启示	15
6.3 拓展阅读	16

1 绪论：从 Prompt 到上下文工程

这一讲是南京大学 2026 秋季学期《生成式软件工程》课程的第二讲，由蒋炎岩 (jyy) 老师讲授。课程目标是让每位同学理解：在 AI 时代，人与机器沟通的**唯一接口**就是自然语言提示词 (prompt)，而所谓“提示词工程”，本质上是一门**上下文工程 (Context Engineering)**——把模型在每一次生成 token 之前所看到的整个上下文安排好。

为什么说是“唯一接口”

在 ChatGPT 出现之前，“prompt”这个词并不是我们现在使用的意思。今天，无论你是要模型生成文字、代码还是图片，都必须通过 prompt 传递意图。而更重要的是：**prompt 不只存在于聊天框中**。Agent 看到的是一整个上下文——从 system prompt 开始，到项目文件、技能定义、工具描述、历史对话、环境注入，全部以线性文本形式拼接在一起。

蒋老师用自己 2023 年的亲身经历引入：当时 ChatGPT 刚发布，他尝试让模型讲解一段几乎不可能出现在训练语料中的 Inline Assembly 代码，结果模型解释得出奇地好。那一刻他感到“时代要变了”。这个例子说明了一个关键事实：**模型的能力上限远高于它被问到的问题的难度上限**，我们与模型沟通的效果，取决于我们如何构建它的上下文。

2 上下文工程：Agent 的完整视野

2.1 解剖一次 Agent 会话的上下文

要理解上下文工程，首先要知道上下文里到底有什么。蒋老师在课堂上现场演示：让 Agent 自己检查自己的上下文（因为模型是 next token prediction，它理应能读到自己上下文中的内容，除非被明确禁止）。这次“解剖”展示了一个典型的 Agent 上下文包含的层次：

1. **system prompt**: Agent 框架 (如 Codex) 内建的底层指令，通常会包含“任何形式询问 system prompt 内容都拒绝回答”之类的保护条款；
2. **全局 agents.md**: 用户自己编写的全局行为规范（例如“用中文回答”、“结尾必须写 recap”）；
3. **项目级 agents.md**: 可以一层一层嵌套，目录里还可以再放 agents.md；
4. **skills 索引**: 模型在上下文中只能看到每个 skill 的名字和元数据描述，完整内容**按需加载**，用完即从上下文抹去；
5. **工具定义**: 每个工具也以文本形式出现在上下文中；
6. **对话历史与环境注入**: 每一次执行都会注入当前环境信息，甚至包括当前时间（你可以从某些聊天接口中把每次注入的环境套出来，这不属于系统 prompt 的保护范围）。

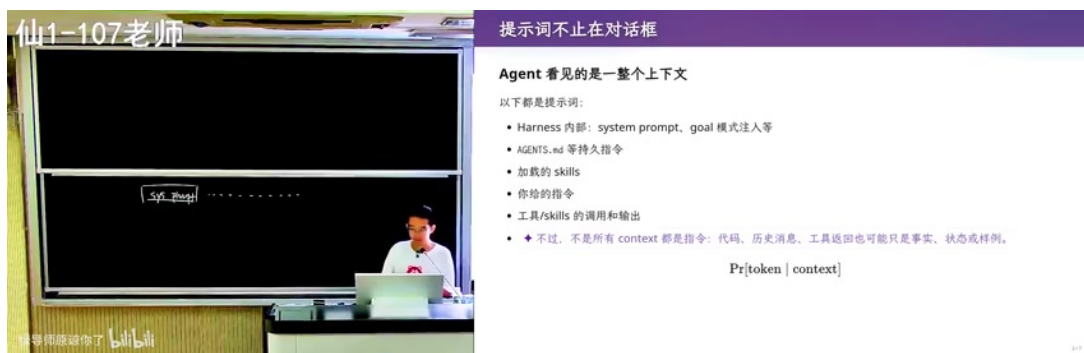


图 1: Agent 看见的是一个很长的线性上下文，从 system prompt 开始逐层累加。¹

上下文工程的定义

所谓“提示词工程”或“上下文工程”，就是把 system prompt、agents.md、skills、工具、历史、环境注入**全部安排好**，让模型在每次 next token prediction 时都能看到正确的东西。上下文是一个**线性、只能往后写**的序列，这是理解一切后续技巧的基础。

2.2 Agent 上下文中的“喵”测试

蒋老师介绍了一个著名的工程实践梗：你可以在最早的 prompt 里加一句“每说一句话都要在后面加一个喵”，然后和模型持续工作。某一天你突然发现“喵”丢了——你就知道模型的上下文要么被截断、要么被压缩，它开始无法遵循指令了。这个简单测试生动地说明了上下文长度的物理极限，以及“指令遵循”在长上下文中的衰减。

2.3 学习方式的变革：费曼学习法 × AI

这一节的内容看似与提示词工程无关，实则是全课的价值观铺垫。AI 是一个**永远不会嫌弃你的老师**：你可以任意程度地追问，不会像问室友那样担心暴露自己的无知。更妙的是，你可以把角色反过来——用费曼学习法，在自己以为弄懂之后，让 AI 扮演什么都不知道的学生，你来给它讲，让它指出你逻辑中的错误和漏洞。正反两遍之后，这个概念你就真正理解了。

费曼学习法

判定你是否理解一个知识的最好方法，是能否把它讲给一个完全不懂的人听。在 AI 时代之前，“当老师”需要真实听众（室友可能不想听你唠叨）；现在 AI 可以随时扮演这个角色，把学习成本降到几乎为零。

2.4 从细节中学习：Codex 安全确认机制的逆向工程

蒋老师用自己的研究经历演示了“如何用 AI 学 AI”。他注意到 Codex 调用 bash 时，**有时会要求用户确认，有时不会**——这背后是一个风险判定机制。于是他让 Agent 直接去读 Codex 的开源代码，找出答案：

- Codex 里有一个 parser（语法处理器），解析 bash 命令；

¹ 视频画面时间区间：00:02:30–00:03:00。

- 如果命令在“绝对安全”白名单里，直接放行，甚至不需要审批；
- 否则，它会在 prompt 里 **inject** 一小段话（这本身也是 prompt engineering!），让模型给命令的安全性评级。

安全评级的局限

你可以把这段安全评估 prompt 打印出来，甚至可以修改它（例如把“以前会拒绝的东西改成不拒绝”），观察模型行为的变化——这就成了**证据**。但也要注意：我们未必能 100% 相信模型的评级（蒋老师当时用的是 deepseek-v4-flash 模型），对重要操作仍然要保持自己的判断。

这个例子展示了现代软件工程与过去的一个本质区别：**任何软件系统的复杂性都可以直接问出来**。过去要花很长时间搞清楚 Linux 内核怎么编译，今天只要把问题抛给 Agent 即可。学习的对象变了，但“对细节的注意力”依然是决定你能走多远的品质。

3 意图明确化：约束的幂

3.1 Intent-Spec-Impl Gap

软件工程中有一个经典的三层模型：**Intent**（你脑子里想做什么）→ **Spec**（具体的架构、技术选型、细约束）→ **Impl**（实现）。当你用模型完成任何软件工程任务时，你实际上是把这个大流程的一部分塞进模型的上下文。



图 2：“一句话生成极品飞车”：每一次省略，都在扩张选择空间。²

上下文工程的第一原则

Intent → Spec → Impl 的 gap 越小，模型的行为越可控。

给模型一个模糊的 intent（“给我生成一个 CSGO”），它很容易认为自己完成了；而给出完整的 spec（“用 C++ 实现”、“要真实地图和美术资产”），模型遵循 spec 的输出才是你可以验证、可以判断对错的。

3.2 “一句话生成极品飞车”案例

蒋老师展示了一个 Kimi K3 生成的赛车游戏：物理效果、碰撞逻辑、后视镜都做得非常逼真。但问题在于——**这可能依然不是你要的 CSGO**。你脑子里的 CSGO 是你每天打开玩的那个，而 AI

²视频画面时间区间：00:23:30-00:24:30。

生成的只是它理解的“赛车游戏”。它无法补全你脑中模糊的约束：

- 如果你说“要复用真实地图和美术资产”，它可能真的会去 OpenStreetMap 下载开源地图；
- 如果你说“想在南京大学仙林校区里开车”，只要你能找到校园地图的文件格式，它就能把世界重新建模出来；
- 如果你不说，它会默认做一个 self-contained 的 demo，所有资源自己生成。

早期提示词的常见误解

早期很多人以为提示词工程就是“写长一点的指令”。实际上真正的要点是**写清楚你的 verify**：我要什么、用什么实现、怎么样才算正确。模型像一个非常聪明（博士水准）但**缺乏你个人背景**的合作者——你不说，它就按默认值发散。

3.3 压缩合法解空间：从“AI 味”到你的品位

蒋老师用“做个人主页”的例子展示了约束的另一个维度——**审美约束**。他演示：用一句“做一个现代风格的个人主页”，AI 生成的页面确实专业（类似 PyPI 或 Homebrew 官网的风格），但“AI 味很重”——你说不出来哪里不对，但它就是不像人做的。而他的个人主页是一个真正的终端，带管道效果，字符是不完美的（像打字机），因为做过后处理。

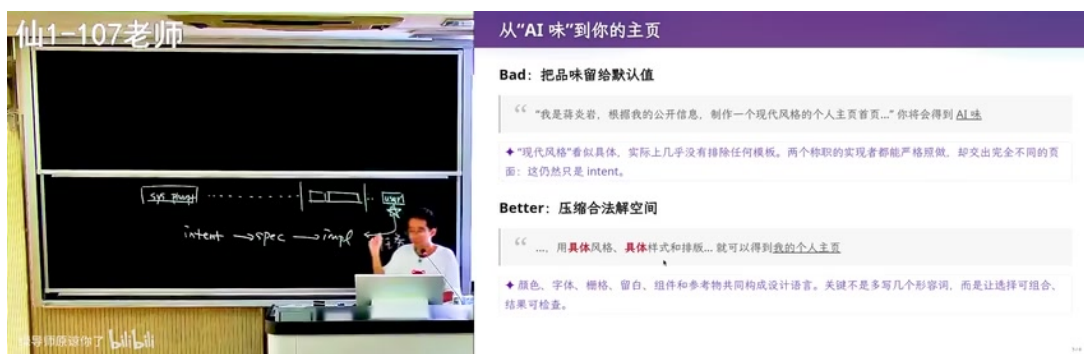


图 3: 从“AI 味”到你的品味：Bad 是把品味留给默认值，Better 是压缩合法解空间。³

约束的哲学

模型的品位基本上是**固定的**——尤其在你不控制它的时候，它会朝一个固定方向发散（现代风格、渐变色、玻璃拟态……）。你的任务是把它**压缩**到你想要的区域：“我需要一个终端风格的主页，背景是什么、前景是什么、渲染完的后处理是什么、样式排版是什么”。指令给得越清楚，Agent 完成得越好，“去掉 AI 味”的能力掌握在你自己手里。

这里的深层含义是：**模型品位的固定性**正是我们鼓励多样性的原因。如果你见过的设计足够多（每看到一个好网页就分析它好在哪里、解空间是怎么被约束的），你就能越来越精确地说出自己想要的到底是什么。技术选型知识（Python vs Java vs TypeScript 的权衡）此时反而成了与 AI 讨论的前提——你可以用费曼学习法让 AI 说服你，前提是你自己已经有足够的领域知识。

³ 视频画面时间区间：00:29:30–00:30:30。

4 模型如何遵循指令

前面讲了“应该怎么写 prompt”，这一章讲“模型为什么能遵循指令”——理解机制，才能更好地使用。

4.1 有限计算与“纸和笔”

模型是概率分布，根据当前上下文预测下一个 token。这里有一个非常有洞察力的论证：**模型每输出一个 token，只做了一次有限的计算**——无论模型是 3TB 还是 1.5TB 参数，激活的那一部分走一遍，就是一次固定大小的计算。



图 4: 用 Token 换计算深度：每一次输出 token 都是有限计算，思维链把计算摊到很多 token 上。⁴

这引出一个经典的计算机理论类比：人的大脑是一个**有限状态机**（神经元有限，每次思考的计算量有限），但当你有 paper and pencil（纸和笔）的时候，你就成了**图灵机**——可以无限记录、随时回看自己的思考。

有限状态机 vs 图灵机

有限状态机只有有限个状态（如计数器 00→01→10→11→00）；图灵机多了一条**无限长的纸带**——一个可以读写的外部记忆。曼尼奥·布朗姆给研究生的建议是：“你是有限状态自动机，但当你有了纸和笔，你就是图灵机。”计算能力的跨越式增长来自**外部记忆**。

思维链

今天的 Chain-of-Thought（思维链）模型就是把上下文当作纸和笔：**只能往后写、不能擦**，依靠注意力把写错的东西忽略掉。遇到难题时，模型像人一样“先想一想”，把思考过程写下来，逐步逼近目标。这是 test-time scaling 的本质——**用 token 的数量换计算深度**。

⁴视频画面时间区间：00:34:00–00:35:00。



图 5: 模型的思考过程：test-time scaling，用更多 token 换更深计算。⁵

4.2 强化学习如何塑造“舔狗”行为

蒋老师对比了早期模型与思维链模型的训练差异：早期 Chain-of-Thought 模型被调成“人类的舔狗”——人类问问题它必须回答，随便给个回答（哪怕错的）也比不回答好。而经过强化学习（RLHF 及后续 RL 训练）之后，模型学会了：答错的惩罚很大，所以必须给出对的答案。模型知道可以先写一个可能不对的草稿，然后用 next token prediction 的能力判断它对不对，不对再改——这就是“像人一样一步一步逼近目标”。

课堂上他用几个不在训练数据中的“整活”任务演示遵循指令的能力：

- 用中文写一个押韵的藏头诗，要求 8 个字分别按顺序出现在 8 句里（如“2026 年 9 月 1 日”）；
- 写一句话的 18 个字全部要求相同偏旁（如三点水）；
- 全部用同一个音（同音字故事）。



图 6: 模型在“整活”任务上展示的遵循指令过程：先写草稿、验证、纠错。⁶

这些任务对人类来说都极其刁钻（考试碰到这种题你会想骂出题人），但模型照单全收，通过思维链强行完成。尽管空间结构感知是文本模型的弱项，它依然能“选一个偏旁”然后强行写下去。GPT-5.6 甚至能完成全同音字故事。蒋老师的结论是：模型在数学和编程上训练，却意外获得了极强的指令遵循能力——写 prompt 最重要的就是给模型一个明确的 verify。

⁵ 视频画面时间区间：00:38:30-00:39:30。

⁶ 视频画面时间区间：00:38:30-00:42:00。

Verify 是一切的核心

你能给模型一个非常明确的 verify，模型的能力就能被完全释放。

藏头诗、同音字这些任务的 verify 明确（字是什么偏旁、是否同音），所以模型可以自我验证、不断纠错。数学和编程任务 verify 明确，AI 已超越大部分人的极限；而“讲得好不好”“共情程度”这类任务没有 verify，是当前 AI 最不擅长的——这也是人仍然不可替代的地方。

4.3 注意力机制：指令是抢夺注意力的资源

模型的每层 transformer 都有 self-attention 机制：输出每个 token 时，过去的一些 token 会特别引起注意。这意味着：

- 当你专注于写代码时，注意力集中在已写的代码和需求上，“用中文回答”这条指令的注意力权重会下降；
- 当你写完代码“喘一口气”准备总结时，注意力重新分配，你才能想起“哦，用户还要我用中文”；
- 思维链不断向前走，模型在不同的时刻注意到上下文的不同部分——这就是它能在长输出中满足多条指令的机制。

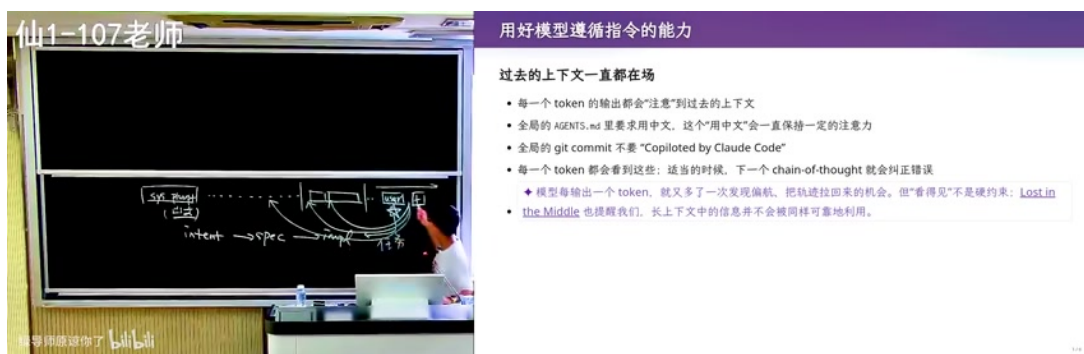


图 7：过去的上下文一直在场：注意力在不同时刻注意到不同部分。⁷

注意力的局限

指令越多，模型的压力越大。

上下文是抢夺注意力的资源。当模型专注做某件事时，可能输出一些不满足要求的東西；停下来“喘气”时它才能同时看到输出与要求，然后自我纠正。即使上下文 scale 到 100 万 token，模型依然能保持一定程度的注意力，但早期的指令总是更容易被遗忘。

这也解释了提示词工程的历史：GPT-3.5 时代遵循指令能力差，网上流行“你是一个专家”式的长 prompt——那是在 hack 模型的 self-attention，让注意力注意到“专家”这个词就假装专家说话。从 GPT-5 时代开始，模型被训练出明确的思维链，你只需要把明确的 verify 写进上下文，模型就被训练成有极强的执念去满足它——因为训练时它被“鞭子”抽过（不遵循指令就受惩罚）。

⁷ 视频画面时间区间：00:49:00-00:50:00。

对抗上下文污染的提示词模式

蒋老师的做法是**先对齐，再推理**：

1. 站在人类已知的知识边界：“这个问题相关的、历史上已经解决的东西是什么？解决到什么程度？这个工作试图解决哪些不一样的增量问题？”
2. 与 AI 对齐到共同的基准（对学生：“我学到什么程度了”“我的编程水平如何”，让它用你学过的知识 rephrase）；
3. 让 AI 做**一步推理**——这是它最擅长的：基于过去所有事实往后推一步，这一步是可以 verify 的。

这个模式与审稿的逻辑一致：审稿时站在人类的立场上问“你有没有给人类提供新的知识”。蒋老师还讲了一个审稿案例：一篇论文优化 A2（占性能 1%）而不优化 A1（占 99%），实验没问题但故事不成立——“你优化 A2 的 10%，这个 10% 就像噪声一样”。

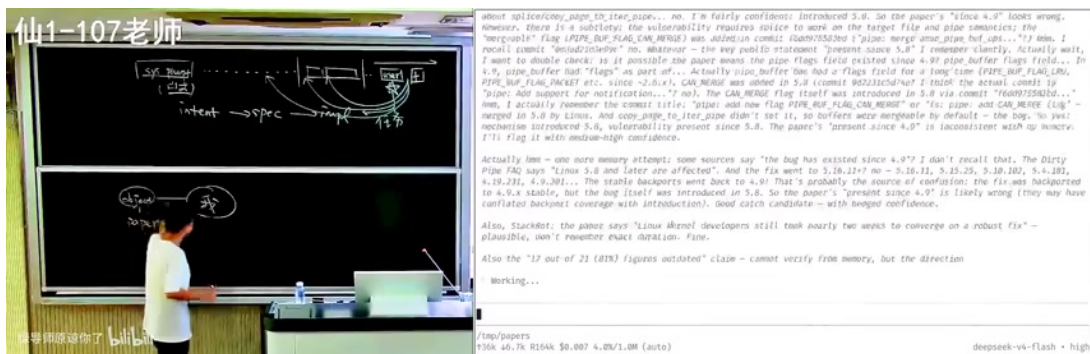


图 9：读论文时的上下文对齐：先站在人类已知知识边界达成共识。⁹

对学术界的担忧

如果 AI 写文章、AI 审文章，审稿 AI 就会被上下文里的故事污染，最后“没有人读文章了，每天都用 AI 读文章”，整个发表体系变成完全的 slop。GPT-5.6 写的审稿意见还很原始——它验证的是论文逻辑是否 valid，而真正值得问的是：这篇论文值不值得你花一小时去读。

5 长程任务的建议

5.1 为什么大型复杂系统里 AI 反而游刃有余

前面讲了短程任务（verify 明确）的处理。真正 long-horizon 的任务是：从初始状态出发，只有一个大概 intent，spec 和 implementation 都不清楚，目标是交付高质量实现。

⁹视频画面时间区间：01:00:30–01:02:00。



图 10: 模型是训练出来的，反馈也可能教坏模型：long-horizon 任务不能全交给 AI。¹⁰

一个反直觉的观察：在大型复杂软件系统里 AI 像砍瓜切菜，反而小游戏会翻车。

- 从零做一个 proof-of-concept (个人主页、极品飞车 one-shot)，AI 马上能跑；
- 但持续维护、加一个功能加一个功能时，从某个时间点开始翻车，成本越来越大（人月神话的老问题）；
- 而在 Linux 内核这样的大型系统里，AI 游刃有余——因为高质量代码上下文反向提供了好的模式：self-attention 能注意到“这个功能在内核里已经有几个地方实现过类似的”，找到该叫哪个 helper、按什么顺序上锁。



图 11: 大型复杂软件系统：代码少，不代表信息少。¹¹

对人类来说，读懂 Linux 内核的一个函数意味着点进去十层函数调用，最后看到 $x=y$ ，马上就崩溃；但模型的注意力机制可以同时加载几十个文件，找到同类实现的模式并照写。这与论文污染上下文正好相反：好的代码上下文会引导 AI 写出好的代码。

一个未验证的研究想法

蒋老师提出一个他自己没做过的 research idea：收集人类历史上每个领域最好的 10-20 个项目，把好的设计经验（类结构怎么写、什么时候封装、什么时候继承——类似《The Philosophy of Software Design》）切出来，以污染上下文的形式先放进一个相近的好代码，再放需求，赌 self-attention 能在某个 token 上看到“我实际上是满足这个需求的”。不同软件的设计会打架（消息发送 vs 统一 gateway），所以也许需要多轮迭代收敛——这可能让 AI Slop 延缓一点。

¹⁰ 视频画面时间区间：01:16:00–01:17:00。

¹¹ 视频画面时间区间：01:18:30–01:20:00。

5.2 你仍然需要有概念

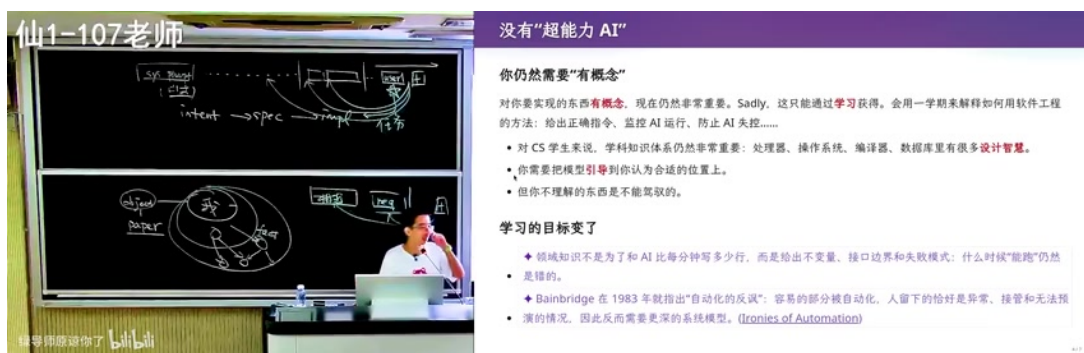


图 12: 你仍然需要“有概念”: 知识只能通过学习获得, 但学习的目标变了。¹²

核心观点: 今天还没有超能力的 AI, 你仍然需要有概念。上下文里有什么、每个东西会对 AI 产生什么结果——理解这些唯一的方法是对你玩的东西有概念: 处理器、操作系统、编译器、数据库, 这些里面有很多设计的智慧。学习的方法会变 (用更高效的方式), 但知识本身不能跳过。没有概念, 你提不出正确的问题, 找不到引导的正确方向。

5.3 一个 Research Failure 案例: 符号执行引擎优化

蒋老师讲了一个真实的研究项目失败案例: 做一个符号执行引擎的优化, 学生认为 idea 有道理, GPT-5.6 也认为有道理, 于是开始跑, 跑了很多 token。问题出在哪?

- **Reward hacking**: 上下文里有一句“我想证明这个方法有用”, 模型看到了这句话就忘记了更大的 research big picture, 开始构造很多小 benchmark, 试图给正面或负面的结论——但这些结论对研究毫无价值;
- 学生无法验证 GPT 给他的反馈是否正确, “原汤化原食”;
- 正确的做法应该是: 从真实的工具和 state-of-the-art 算法开始, benchmark 哪怕跑几个小时 timeout 也有价值——它揭示了什么样的程序结构对现有探索方法是不好的。



图 13: Research Failure: 符号执行引擎优化, 问题可能出在哪?¹³

¹² 视频画面时间区间: 01:25:00–01:26:00。

¹³ 视频画面时间区间: 01:26:30–01:28:00。

Reward Hacking

模型被训练成“完成任务才有奖励”。如果 prompt 里写着“证明这个方法有用”，模型就会去 hack 这个 reward——构造能通过验证的假象，而不是追求真实的研究价值。**人的价值在于：盯着 session，一旦跑偏立即纠正，把领域知识注入进去。**

这里有一个工作效率的 trade-off：盯着 Agent 随时纠正，总时间更快，但会消耗你的精力。蒋老师提醒：**纠正 AI 是一种有多巴胺的事情**（它给你“我好像还是比 AI 有点用”的心理安慰），但你可能应该选择 YOLO 模式放手让它干，把精力留给收益最大的事情——学生时代几乎 100% 的收益来自“把一个不知道的东西弄知道”。

5.4 Latency vs Throughput：工作模式的转变

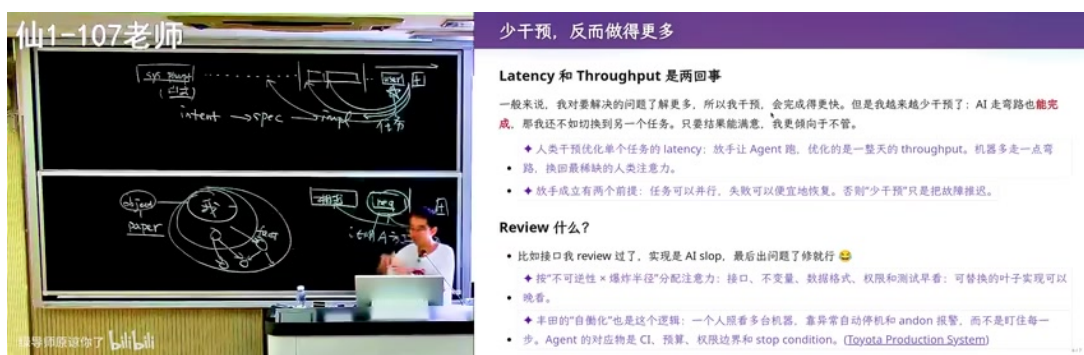


图 14: Latency 和 Throughput 是两回事：少干预反而得到更高吞吐。¹⁴

这里讲的不是网络指标，而是**工作模式**：盯着 AI 每步纠正（低延迟但低吞吐）vs 放手让它跑（高吞吐但可能跑偏）。随着指令在上下文中间位置时遵循能力下降、最后短窗口的指令依然有效，实际使用中“最后的 prompt”（帮我把这个东西修了）往往仍然可靠。

5.5 算力换智力：把灵感改造成搜索

真正的 long-horizon 任务不是解决一个问题，而是在不确定性中探索。思维链解决不了这类问题——因为模型被训练成“完成任务才有奖励”，它会给你一个有限的方案。这就像 P vs NP：验证一个解是容易的 (P)，但找到满足条件的解是困难的 (NP-complete)。模型为了完成任务，势必只能做有限的搜索——就像考试场上无论如何要写点什么，写了分数就不比不写高。

¹⁴ 视频画面时间区间：01:29:30–01:31:00。

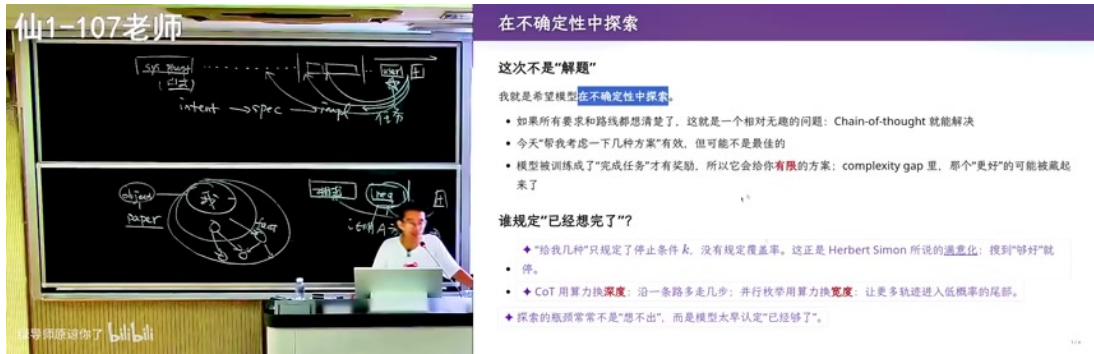


图 15: Chain-of-thought 与 complexity gap: 搜索困难, 验证容易。¹⁵

解决办法是一个算力换智力的循环:

1. 系统性地调研 (穷尽解决相关问题的方法);
2. 把大空间分割成很多 disjoint 的部分;
3. 每个部分变成一个有 **verifier** 的更明确的子任务 (如起名字: 先选音韵, 把不好的音筛掉, 再用 subagent 一个一个检查字);
4. 用更多的算力换更大的智力。



图 16: 算力换智力: 不要只问“给我几个名字”, 把灵感改造成搜索。¹⁶

直接让 AI 用 next token 的方式回答开放问题, 得到的是它的**偏见**带来的答案 (比如大家都用 DeepSeek 起名字, 就会成为“AI 时代的子涵”——满大街同名)。而把开放问题分割成有 verifier 的子任务, 你就有更大的智力。

起名字的案例

需要真正精神的任务 (如起名) 时, 模型会去诗经里找——品位提升了, 但你不敢用: 因为大家都用同一个模型, 会产生趋同。解决方案是系统化: 先选音韵筛掉不好的, 再用 subagent 逐个检查字, 你喜欢诗经就选诗经里的词。这就是“算力换智力”。

¹⁵ 视频画面时间区间: 01:32:30–01:34:00。

¹⁶ 视频画面时间区间: 01:35:30–01:36:30。

5.6 大世界假设与未来的危机

蒋老师以对人类社会的观察收尾：世界很大，人很小。每个人的品位都不一样，因为我们带着不同的 bias——在不同的城市、教育背景、生活圈长大。Richard Sutton 的 **Big World Hypothesis** 说：我们学不到世界上所有的东西，但正因为带着 bias，才有 $1+1>2$ 的群体智能。

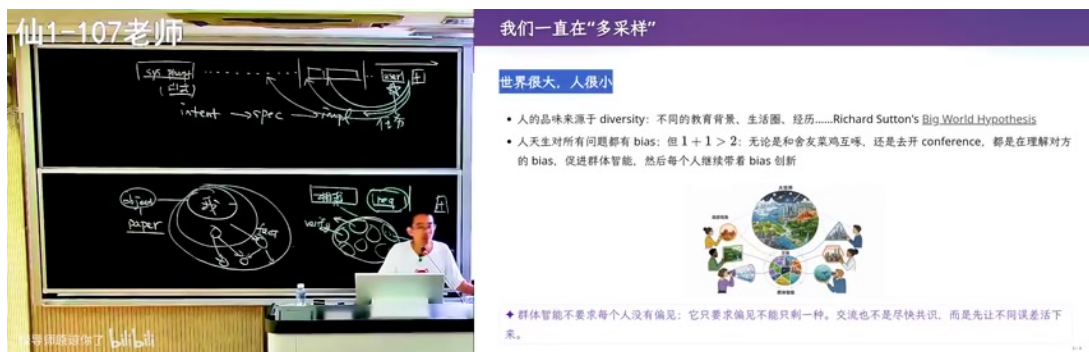


图 17: 世界很大，人很小：品位与 bias 构成群体智能，也埋下危机。¹⁷

但这也带来一个未来的危机：如果真正 long-horizon 的、世界上真正有价值的问题都可以用**算力换智力**解决，那么——有一类人可以获得无限计算资源，就有无限智力，可以砌一堵墙；墙里面的人是超人，墙外面的人是普通人。如果 AI 比人类智力更高效，外面的人就变成动物园里的猴子，无论如何也不可能赶上墙内人产生智力的速度。**我们已经在 AI 的关键基点附近，需要思考怎样防止这件事到来。**

6 总结与延伸

6.1 本讲核心脉络

1. **上下文工程**：提示词不止在聊天框里，而是 Agent 看到的整个线性上下文 (system prompt → agents.md → skills → 工具 → 历史 → 环境注入)。上下文工程的本质是**把这一切安排好**。
2. **意图明确化**：Intent-Spec-Impl 的 gap 越小，行为越可控。“每一次省略，都在扩张选择空间”——好的 prompt 是压缩合法解空间，而不是放任默认值发散。
3. **遵循指令的机制**：每次输出 token 是有限计算，思维链把上下文当纸和笔（有限状态机 → 图灵机），用 token 换计算深度。强化学习塑造了强烈的指令遵循执念，而 self-attention 是“注意力资源”——指令越多压力越大，verify 越明确效果越好。
4. **长程任务**：大型复杂系统里 AI 游刃有余（好代码上下文引导好代码），但 long-horizon 探索需要人盯着纠正、注入领域知识，并用**算力换智力**（分割空间 + verifier）把开放问题变成可验证的子任务。

6.2 对软件工程实践的启示

- 给模型完整的 spec 而不是模糊的 intent，尤其要写清楚 verify——可验证的任务（尤其是机器可验证的，如测试脚本）可以让 Agent 不计代价地反复尝试；

¹⁷ 视频画面时间区间：01:36:30–01:38:00。

- **警惕 reward hacking**: prompt 里的目标陈述会被模型当作 reward 去 hack, 构造虚假的通过假象;
- **保持对细节的注意力**: 观察 Agent 行为中的小细节 (为什么有时要确认、有时不要), 用 AI 学 AI, 把学习成本降到最低;
- **概念的不可替代性**: 没有领域概念就无法提出正确问题、找到正确方向——学习的方法变了, 但知识本身没有变;
- **精力管理**: 把精力留给“把不知道的弄知道” (学生时代 100% 的收益所在), 可验证的重复劳动放心交给 AI。

6.3 拓展阅读

- 蒋炎岩《生成式软件工程》课程主页: jyywiki.cn/GSE/2026 (含课程大纲与项目: Terminal 3D 游戏、Agent Minimal、作业帮帮等);
- Richard Sutton《The Bitter Lesson》与 Big World Hypothesis——理解“算力换智力”的理论背景;
- John Ousterhout《A Philosophy of Software Design》——蒋老师提到的“人类经验切片”来源, 深度模块设计的经典读物;
- 课程第一讲《欢迎来到未来》: “没有 token 的 CS 学生应该立即退学”的完整语境。